CUNEF UNIVERSIDAD

Relational databases · Databases practice

ER → Logical: summary

1. **Entities and attributes**: Entity → table, identifier → primary key · Composite and derived attributes · Multivalued attribute → its own table
2. **Binary relationships**: 1:N relationship → foreign key on the N side · M:N relationship → its own table · 1:1 relationship → the key passes to one side · Participation → NOT NULL on the foreign key
3. **Special cases**: Weak entity → owner's key + partial key · Unary (recursive) relationship · Ternary relationship → a table with three FKs
4. **Hierarchies**: Hierarchy: the three strategies · Choosing a hierarchy strategy

Entities and attributes

Entity → table, identifier → primary key

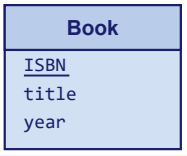
Every entity type becomes a table with the same name, and its identifier becomes the table's primary key.

- Rule 1: **entity** → **table**, named after the entity.
- Rule 2: **attribute** → **column** of that table.
- Rule 3: **identifier** → **primary key**.

Resulting tables

Book (ISBN, title, year)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** The entity `Book` with ISBN, title and year becomes `Book(ISBN, title, year)`. One row per book, and the ISBN identifies it.

> **Common mistake.** Adding an artificial id column when the entity already has an identifier. If the model says the ISBN identifies a book, the ISBN is the primary key.

Composite and derived attributes

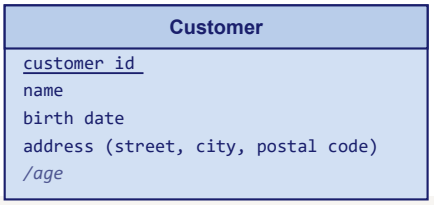
A composite attribute is split into one column per part. A derived attribute is left out, because it can be computed.

- **Composite** → its parts become columns; the composite name itself disappears.
- **Derived** → no column; compute it in queries.

Resulting tables

Customer (customer id, name, birth date, street, city, postal code)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** `Customer(customer id, name, birth date, address(street, city, postal code), /age)` becomes `Customer(customer id, name, birth date, street, city, postal code)`.

> **Common mistake.** Keeping an `age` column next to `birth date`. After a year the stored age is wrong, while the birth date is still right.

Multivalued attribute → its own table

A multivalued attribute becomes a new table. Its primary key is the owner's key plus the value.

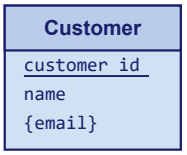
- New table, usually named owner + attribute (`CustomerEmail`).

- Columns: the owner's key (**PK and FK**) + the value (**PK**).
- The owner's table keeps its other attributes and loses the multivalued one.

Resulting tables

Customer (**customer id**, name)
CustomerEmail (***customer id***→Customer, **email**)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** `Customer(customer id, name, {email})` becomes `Customer(customer id, name)` and `CustomerEmail(customer id → Customer, email)`.

> **Common mistake.** Adding the columns email1, email2, email3. That sets an arbitrary limit, leaves NULLs and makes searching for an e-mail look in three places.

Binary relationships

1:N relationship → foreign key on the N side

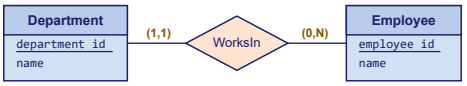
A one-to-many relationship gets no table of its own. The key of the "one" side travels to the "many" side as a foreign key.

- Rule 5: **1:N → FK on the N side**, referencing the 1 side.
- The FK is **NOT NULL** when the minimum at the 1 side is 1 (see "Participation").

Resulting tables

Department (**department id**, name)
Employee (**employee id**, name, *department id*→Department)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** Department (1,1) — WorksIn — (0,N) Employee becomes `Employee(employee id, name, department id → Department)`.

> **Common mistake.** Putting the foreign key on the "one" side, such as an employee id in `Department`. It can only hold one employee per department.

M:N relationship → its own table

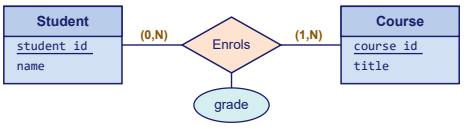
A many-to-many relationship becomes a new table. Its primary key combines the keys of both entities, and each of them is also a foreign key.

- Rule 4: **M:N → table** named after the relationship.
- PK = **both foreign keys together**.
- The relationship's attributes become columns of this table.

Resulting tables

Student (**student id**, name)
Course (**course id**, title)
Enrols (***student id***→Student, ***course id***→Course, grade)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** Student (0,N) — Enrols(grade) — (1,N) Course becomes `Enrols(student id → Student, course id → Course, grade)`.

> **Common mistake.** Making only one of the two foreign keys the primary key. Then a student could appear in a single course only.

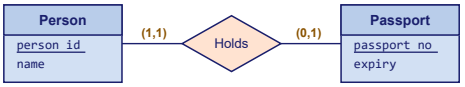
1:1 relationship → the key passes to one side

In a one-to-one relationship the key of either side can pass to the other. With (0,1)/(1,1), the course rule passes it to the optional side.

- Rule 6: **1:1 → the key of either side passes to the other**.
- Rule 7: with (0,1)/(1,1), **the key passes to the optional side** (the one with min 0 at the opposite end).
- Use the switch below to see both accepted designs.

Resulting tables

Person (<u>**person id**</u>, name, *passport no*→Passport?)
Passport (<u>**passport no**</u>, expiry)



Underlined: primary key. Italic with →: foreign key. ? marks a foreign key that can be NULL.

> **Example.** Person (1,1) — Holds — (0,1) Passport becomes `Person(person id, name, passport no → Passport)`, with the FK allowed to be NULL.

> **Common mistake.** Merging both entities into one table. It is sometimes acceptable, but it is not the course rule, and it leaves empty columns for every person without a passport.

Participation → NOT NULL on the foreign key

The minimum cardinality decides whether a foreign key can be NULL. Read it across the relationship, at the end the FK points to.

- Min **1** at the referenced side → FK **NOT NULL**.
- Min **0** at the referenced side → FK **can be NULL** (shown with ? in the notation).
- Primary-key columns are always NOT NULL.

Resulting tables

Customer (<u>**customer id**</u>, name)
Order (<u>**order id**</u>, date, *customer id*→Customer, *courier id*→Courier?)
Courier (<u>**courier id**</u>, name)



Underlined: primary key. Italic with →: foreign key. ? marks a foreign key that can be NULL.

> **Example.** Customer (1,1) — Places — (0,N) Order and Courier (0,1) — Delivers — (0,N) Order become `Order(order id, date, customer id → Customer NOT NULL, courier id → Courier NULL)`.

> **Common mistake.** Reading the cardinality on the wrong side. The card next to `Order` says how many orders a customer has. It decides nothing about the FK in `Order`.

Special cases

Weak entity → owner's key + partial key

A weak entity cannot be identified on its own. Its primary key is the owner's key (also a foreign key) plus its partial key.

- PK = **owner key (FK)** + **partial key**.
- The FK is always NOT NULL, because it is part of the PK.
- Deleting the owner usually deletes its weak rows (ON DELETE CASCADE).

Resulting tables

Building (<u>**building id**</u>, address)
Room (<u>***building id***</u>→Building, <u>**number**</u>, floor)



Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.

> **Example.** Building (1,1) ═ Contains ═ (1,N) Room(number, floor) becomes `Room(building id → Building, number, floor)`.

> **Common mistake.** Using the partial key alone as the primary key. Two buildings with a room 101 would then collide.

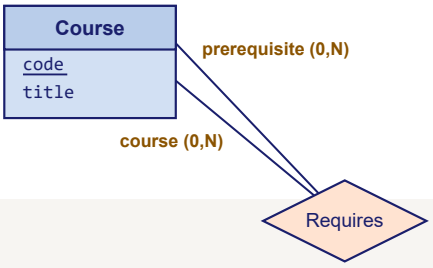
Unary (recursive) relationship

When an entity relates to itself, apply the usual 1:N or M:N rule. The foreign keys point to the same table and are named after the roles.

- Unary 1:N → **a FK to the same table**, named after the role.
- Unary M:N → **a table with two FKs** to the entity, one per role.

Resulting tables

Course (<u>**code**</u>, title)
Requires (<u>***prerequisite***</u>→Course, <u>***course***</u>→Course)



Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.

> **Example.** Course (0,N) as prerequisite — Requires — (0,N) as course becomes `Requires(prerequisite → Course, course → Course)`.

> **Common mistake.** Creating a second table for the same entity, such as `Mentor` next to `Person`. A mentor is a person, so the role is a column, not a table.

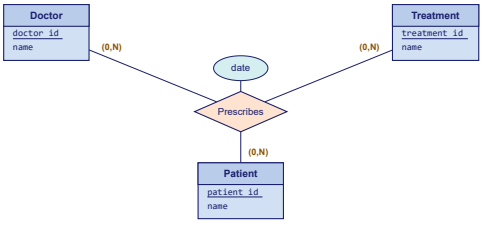
Ternary relationship → a table with three FKs

A relationship among three entities always becomes its own table, with a foreign key to each of them.

- Ternary → **table** with **one FK per entity**.
- PK: the FKs of the "many" ends (all three if every end is N).

Resulting tables

Doctor (<u>**doctor id**</u>, name)
Treatment (<u>**treatment id**</u>, name)
Patient (<u>**patient id**</u>, name)
Prescribes (<u>***doctor id***</u>→Doctor, <u>***treatment id***</u>→Treatment, <u>***patient id***</u>→Patient, date)



Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.

> **Example.** Doctor, Patient, Treatment — Prescribes(date) becomes `Prescribes(doctor id → Doctor, patient id → Patient, treatment id → Treatment, date)`.

> **Common mistake.** Splitting the ternary into three binary relationships. You lose which doctor prescribed which treatment to which patient.

Hierarchies

Hierarchy: the three strategies

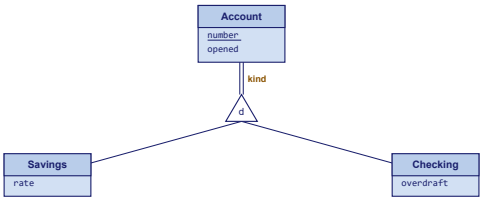
A generalization (supertype and subtypes) can be turned into tables in three ways. Use the switch below to see the tables each one produces.

- Supertype + subtypes: no NULLs; reading a full row needs a join.
- Single table: no joins; NULLs in the subtype columns.
- Subtypes only: no joins and no NULLs, but the supertype cannot be referenced as a whole.

Resulting tables

Account (<u>number</u>, opened)
Savings (<u>*number*</u>→Account, rate)
Checking (<u>*number*</u>→Account, overdraft)

Underlined: primary key. Italic with →: foreign key. **?** marks a foreign key that can be NULL.



> **Example.** Account(number, opened) is either a Savings(rate) or a Checking(overdraft) account, so the hierarchy is total and disjoint.

> **Common mistake.** Choosing "subtypes only" for a partial hierarchy. Accounts that are neither savings nor checking would have nowhere to go.

Choosing a hierarchy strategy

All three strategies are correct. Pick one by how the data is used: shared relationships, how many subtype attributes there are, and whether the hierarchy is total.

Trade-offs of the three strategies

Strategy	NULLs	Joins to read a row	Needs a total hierarchy
Supertype + subtypes	No	Yes (supertype ⋈ subtype)	No
Single table	Yes, in the subtype columns	No	No
Subtypes only	No	No (but a UNION to list them all)	Yes

> **Common mistake.** Leaving out the discriminator in the single-table strategy. Without it a row with every subtype column NULL cannot be classified.